

Report

Deploying a Three-Tier Web Application on AWS

Course: Cloud Computing / AWS

Date: May 4, 2026

Contents

1	Introduction	2
2	Objectives	2
3	Architecture Overview	2
4	Step 1: VPC Creation	3
5	Step 2: Subnet Configuration	4
6	Step 3: Internet Gateway and Route Tables	5
7	Step 4: Presentation Tier (Frontend)	5
8	Step 5: Application Tier (Backend)	6
9	Step 6: Data Tier (Amazon RDS MySQL)	6
9.1	Database Operations	7
10	Step 7: Load Balancer Configuration	7
11	Final Working Output	8
12	Challenges Faced	8

1 Introduction

This report explains the deployment of a three-tier web application on Amazon Web Services (AWS). The project consists of three layers: Presentation Tier, Application Tier, and Data Tier. Networking components such as VPC, subnets, route tables, internet gateway, security groups, and load balancer were configured successfully.

2 Objectives

- Create a custom VPC and subnets.
- Deploy frontend server in a public subnet.
- Deploy backend server in a private subnet.
- Create and connect MySQL RDS database.
- Configure load balancer for traffic handling.
- Demonstrate complete three-tier communication.

3 Architecture Overview

The application follows a standard three-tier architecture:

1. Presentation Tier: Frontend web server.
2. Application Tier: Backend Node.js server.
3. Data Tier: MySQL database on Amazon RDS.

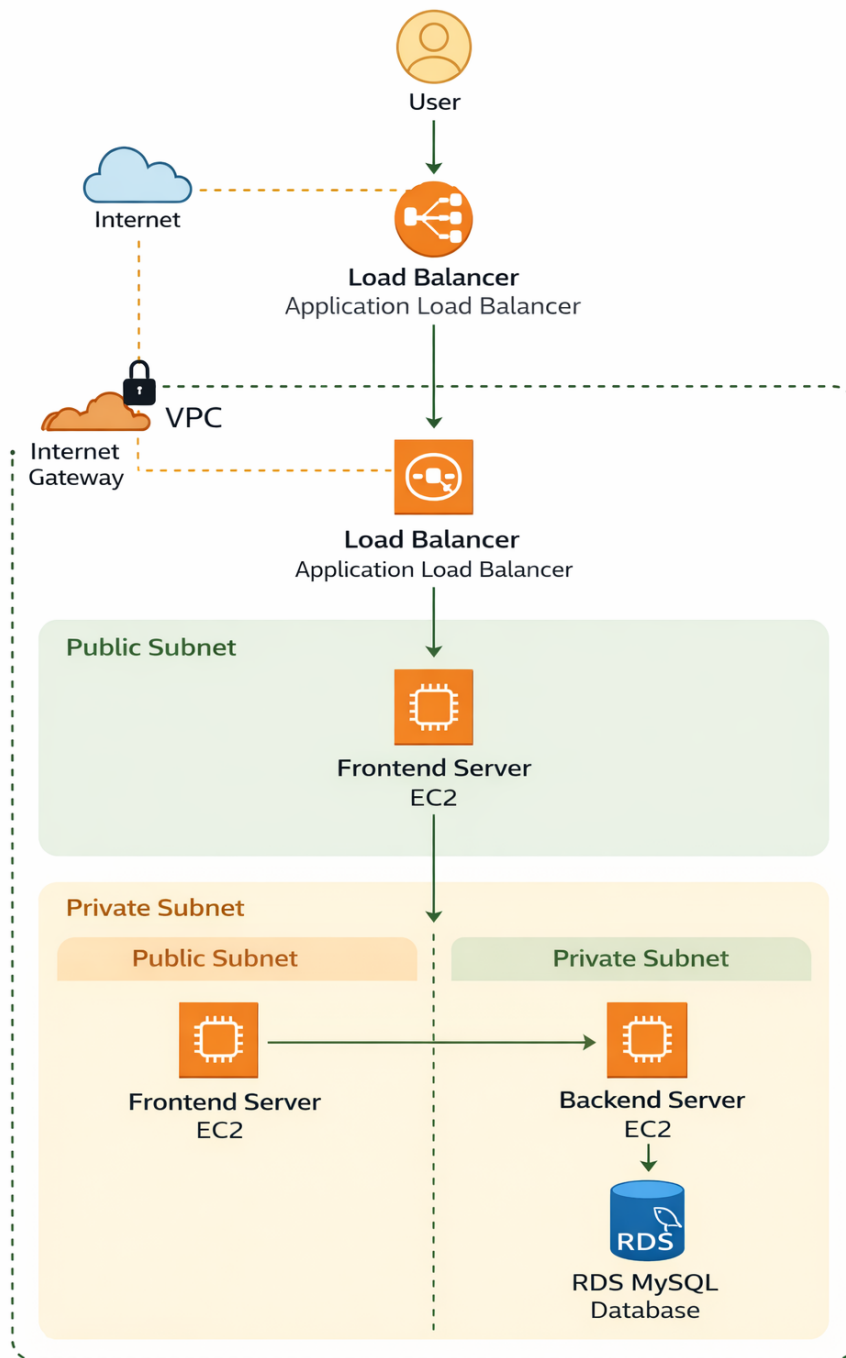


Figure 1: Three-Tier Architecture

4 Step 1: VPC Creation

A custom VPC named `Assignment-vpc` was created with CIDR block `10.0.0.0/16`.

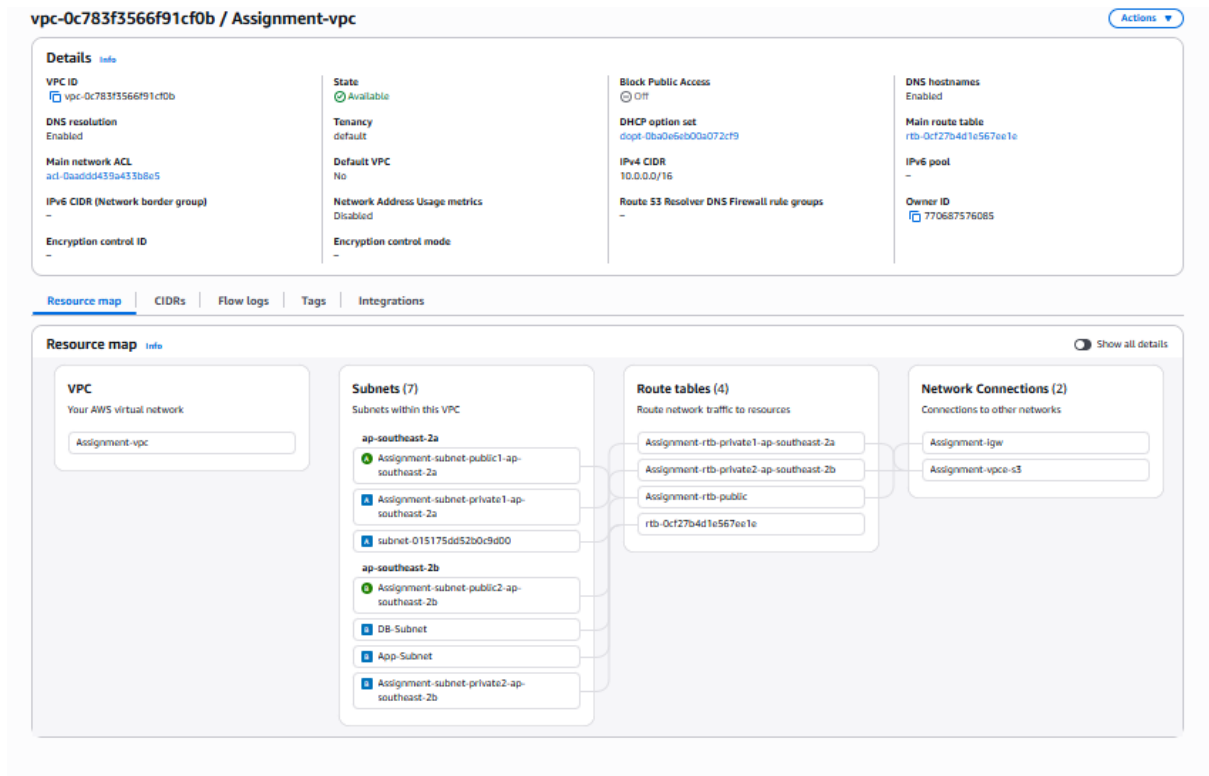


Figure 2: Custom VPC Created

5 Step 2: Subnet Configuration

Multiple subnets were created:

- Public Subnet 1: 10.0.0.0/20
- Public Subnet 2: 10.0.16.0/20
- App Subnet: 10.0.33.0/24
- DB Subnet: 10.0.34.0/24

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIL
DB-Subnet	subnet-077a95ccc457222e7	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.34.0/24	--	--
-	subnet-0e76affc2f30022c8	Available	vpc-08358af8649867354	Off	172.31.0.0/20	--	--
App-Subnet	subnet-0fef9f6c86c75644	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.33.0/24	--	--
Assignment-subnet-public1-ap-southea...	subnet-0c0bba2e3ae13d2ca	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.0.0/20	--	--
Assignment-subnet-private1-ap-sourhe...	subnet-06f0092bcae3c883	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.128.0/20	--	--
-	subnet-02da7f5fe84d02eeb	Available	vpc-08358af8649867354	Off	172.31.16.0/20	--	--
-	subnet-015175dd52b0c9d00	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.32.0/24	--	--
Assignment-subnet-public2-ap-southea...	subnet-0af9d9d6e98b4cde3	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.16.0/20	--	--
-	subnet-0434406833cfb82cc	Available	vpc-08358af8649867354	Off	172.31.32.0/20	--	--
Assignment-subnet-private2-ap-sourhe...	subnet-001dd3b07a3b644db	Available	vpc-0c783f3566f91cf0b Assign...	Off	10.0.144.0/20	--	--

Figure 3: Created Subnets

6 Step 3: Internet Gateway and Route Tables

An Internet Gateway was attached to the VPC. Public route table was configured with route 0.0.0.0/0 to the Internet Gateway.

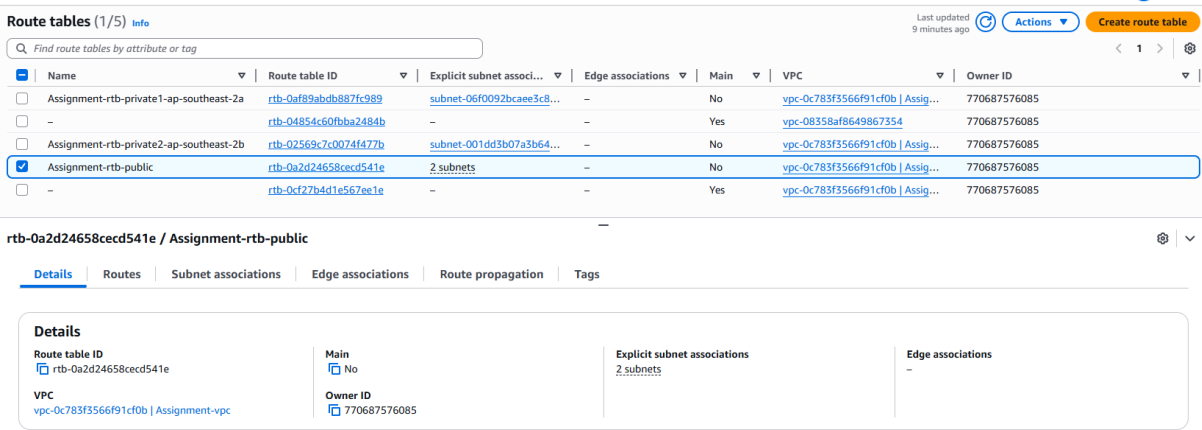


Figure 4: Route Table Configuration

7 Step 4: Presentation Tier (Frontend)

An EC2 instance was launched in the public subnet. Apache web server was installed and a sample frontend page was deployed.

Frontend Features:

- Welcome message
- Call Backend button
- Hosted on EC2 instance

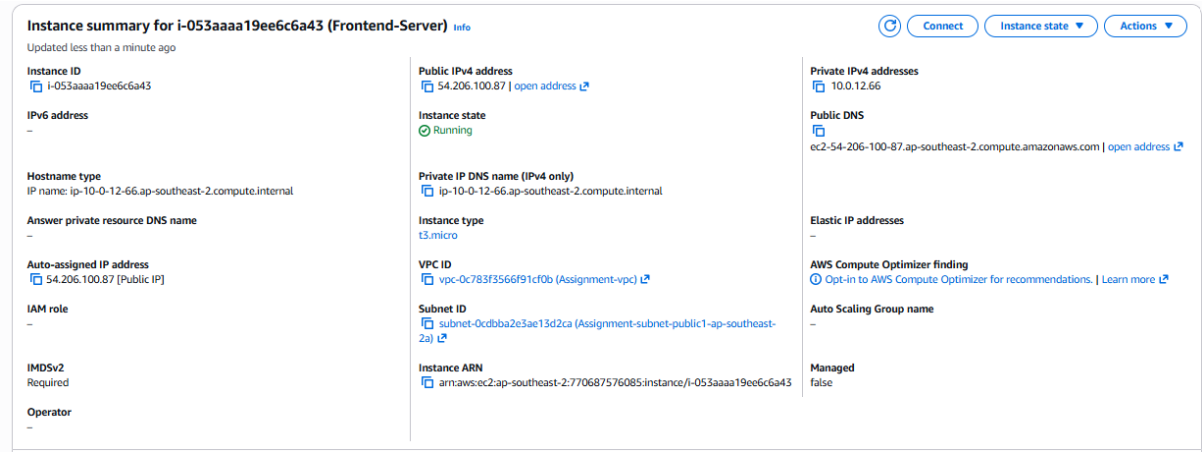


Figure 5: Frontend EC2 Instance

Welcome to My 3-Tier App

Frontend Server Running

Call Backend

Figure 6: Frontend Web Application

8 Step 5: Application Tier (Backend)

A backend EC2 instance was deployed. Node.js server was installed and configured to respond on port 3000.

Backend Response:

Hello from Backend Server

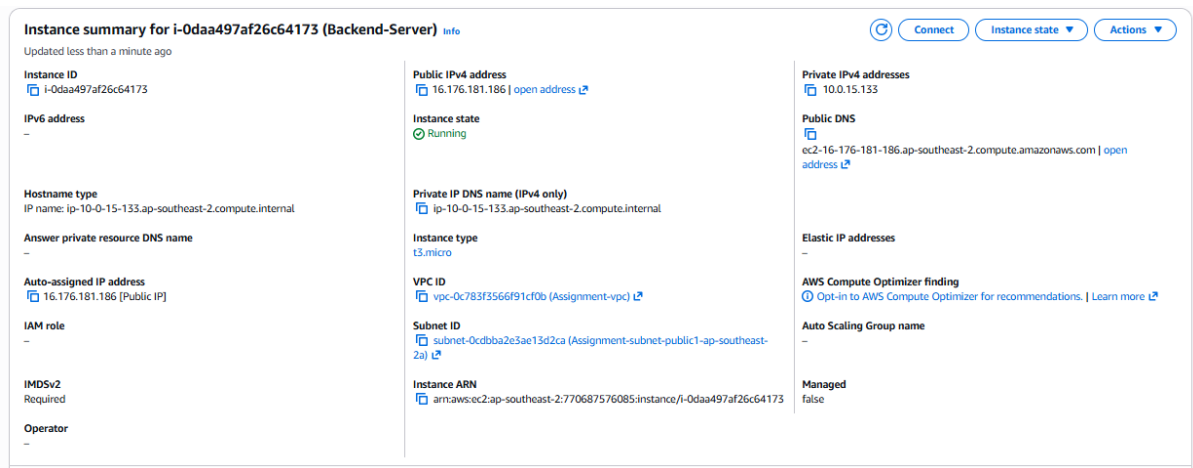


Figure 7: Backend EC2 Instance

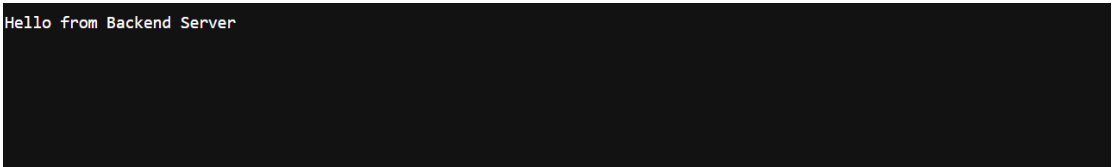


Figure 8: Backend Node.js Server Running

9 Step 6: Data Tier (Amazon RDS MySQL)

An Amazon RDS MySQL database named `assignment-db` was created. Connectivity and security rules were configured successfully.

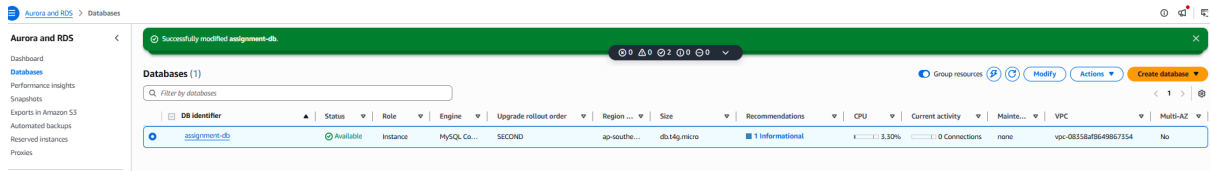


Figure 9: RDS MySQL Database

9.1 Database Operations

The following SQL commands were executed:

```
CREATE DATABASE assignmentdb;
USE assignmentdb;
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50)
);
INSERT INTO users(name) VALUES ('Mohsan');
SELECT * FROM users;
```

```
mysql> SELECT * FROM users;
+----+-----+
| id | name |
+----+-----+
| 1  | Mohsan |
+----+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 10: Database Query Result

10 Step 7: Load Balancer Configuration

An Application Load Balancer (ALB) was created. It forwards incoming HTTP traffic to the frontend EC2 instance using a target group.

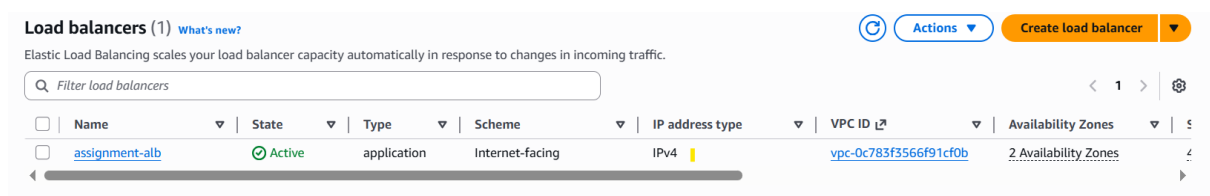


Figure 11: Application Load Balancer

11 Final Working Output

The complete application was tested using the ALB DNS URL. Frontend loaded successfully and backend response was displayed.

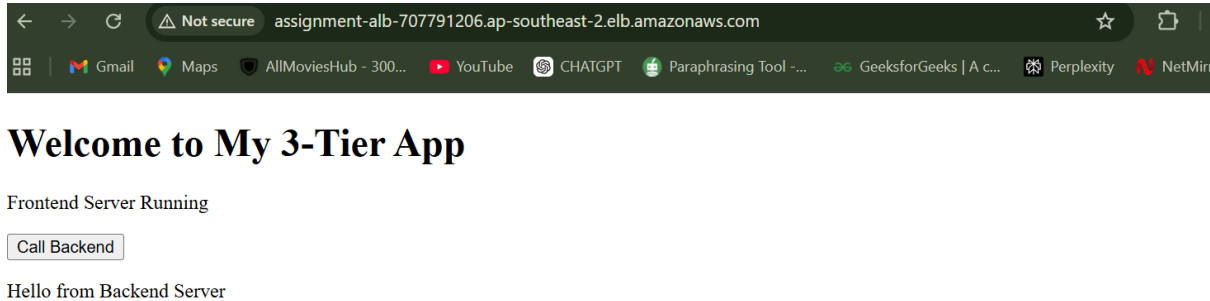


Figure 12: Final Three-Tier Application Output

12 Challenges Faced

- CIDR block overlap during subnet creation.
- Internet Gateway attachment confusion.
- Backend server port conflicts.
- RDS security group and public access issues.
- DNS endpoint troubleshooting.